Morgan Stanley

Q. Two Sum

Given an array of integers, return **indices** of the two numbers such that they add up to a specific target.

You may assume that each input would have ***exactly*** one solution, and you may not use the *same* element twice.

Example:

```
Given nums = [2, 7, 11, 15], target = 9,



Because nums[0] + nums[1] = 2 + 7 = 9,

return [0, 1].
```

Q. Best Team With No Conflicts

You are the manager of a basketball team. For the upcoming tournament, you want to choose the team with the highest overall score. The score of the team is the **sum** of scores of all the players in the team.

However, the basketball team is not allowed to have **conflicts**. A **conflict** exists if a younger player has a **strictly higher** score than an older player. A conflict does **not** occur between players of the same age.

Given two lists, `scores` and `ages`, where each `scores[i]` and `ages[i]` represents the score and age of the `i`th player, respectively, return *the highest overall score of all possible basketball teams*.

Example 1:

```
Input: scores = [1,3,5,10,15], ages = [1,2,3,4,5]

Output: 34
```

```
Explanation: You can choose all the players.
```

Example 2:

```
Input: scores = [4,5,6,5], ages = [2,1,2,1]

Output: 16

Explanation: It is best to choose the last 3 players. Notice that you are allowed
to choose multiple people of the same age.
```

Example 3:

```
Input: scores = [1,2,3,5], ages = [8,9,10,1]

Output: 6

Explanation: It is best to choose the first 3 players.
```

Constraints:

- `1 <= scores.length, ages.length <= 1000`
- `scores.length == ages.length`
- `1 <= scores[i] <= 10^6`
- `1 <= ages[i] <= 1000`

Q. LRU Cache

Design and implement a data structure for [Least Recently Used (LRU) cache](). It should support the following operations: `get` and `put`.
`get(key)` - Get the value (will always be positive) of the key if the key exists in the cache, otherwise return -1.
`put(key, value)` - Set or insert the value if the key is not already present. When the cache reached its capacity, it should invalidate the least recently used item before inserting a new item.
The cache is initialized with a **positive** capacity.

Follow up:
Could you do both operations in **O(1)** time complexity?

Example:

```
LRUCache cache = new LRUCache( 2 /* capacity */ );


cache.put(1, 1);

cache.put(2, 2);

cache.get(1);       // returns 1

cache.put(3, 3);    // evicts key 2

cache.get(2);       // returns -1 (not found)

cache.put(4, 4);    // evicts key 1

cache.get(1);       // returns -1 (not found)

cache.get(3);       // returns 3

cache.get(4);       // returns 4
```

Q. Best Time to Buy and Sell Stock

Say you have an array for which the ith element is the price of a given stock on day *i*.

If you were only permitted to complete at most one transaction (i.e., buy one and sell one share of the stock), design an algorithm to find the maximum profit.

Note that you cannot sell a stock before you buy one.

Example 1:

```
Input: [7,1,5,3,6,4]

Output: 5

Explanation: Buy on day 2 (price = 1) and sell on day 5 (price = 6), profit = 6-1
= 5.

             Not 7-1 = 6, as selling price needs to be larger than buying price.
```

Example 2:

```
Input: [7,6,4,3,1]

Output: 0
```

```
Explanation: In this case, no transaction is done, i.e. max profit = 0.
```

Q. Binary Tree Zigzag Level Order Traversal

Given a binary tree, return the *zigzag level order* traversal of its nodes' values. (ie, from left to right, then right to left for the next level and alternate between).

For example:
Given binary tree `[3,9,20,null,null,15,7]`,

```
    3
   / \
  9  20
    /  \
   15   7
```

return its zigzag level order traversal as:

```
[
  [3],
  [20,9],
  [15,7]
]
```

Q. Maximum Subarray

Given an integer array `nums`, find the contiguous subarray (containing at least one number) which has the largest sum and return its sum.
Example:

```
Input: [-2,1,-3,4,-1,2,1,-5,4],
```

```
Output: 6

Explanation: [4,-1,2,1] has the largest sum = 6.
```

Follow up:

If you have figured out the O(*n*) solution, try coding another solution using the divide and conquer approach, which is more subtle.

Q. Pow(x, n)

Implement pow(*x*, *n*), which calculates *x* raised to the power *n* (x^n).

Example 1:

```
Input: 2.00000, 10

Output: 1024.00000
```

Example 2:

```
Input: 2.10000, 3

Output: 9.26100
```

Example 3:

```
Input: 2.00000, -2

Output: 0.25000

Explanation: 2^-2 = 1/2^2 = 1/4 = 0.25
```

Note:

- $-100.0 < x < 100.0$
- *n* is a 32-bit signed integer, within the range $[-2^{31}, 2^{31} - 1]$

Q. **Letter Combinations of a Phone Number**

Given a string containing digits from `2-9` inclusive, return all possible letter combinations that the number could represent. Return the answer in **any order**.

A mapping of digits to letters (just like on the telephone buttons) is given below. Note that 1 does not map to any letters.

Example 1:

```
Input: digits = "23"

Output: ["ad","ae","af","bd","be","bf","cd","ce","cf"]
```

Example 2:

```
Input: digits = ""

Output: []
```

Example 3:

```
Input: digits = "2"

Output: ["a","b","c"]
```

Constraints:

- `0 <= digits.length <= 4`
- `digits[i]` is a digit in the range `['2', '9']`.

Q. **Find the Duplicate Number**

Given an array of integers `nums` containing `n + 1` integers where each integer is in the range `[1, n]` inclusive.

There is only **one repeated number** in `nums`, return *this repeated number*.

You must solve the problem **without** modifying the array `nums` and uses only constant extra space.

Example 1:

```
Input: nums = [1,3,4,2,2]

Output: 2
```

Example 2:

```
Input: nums = [3,1,3,4,2]

Output: 3
```

Constraints:

- $1 <= n <= 10^5$
- `nums.length == n + 1`
- `1 <= nums[i] <= n`
- All the integers in `nums` appear only **once** except for **precisely one integer** which appears **two or more** times.

Follow up:

- How can we prove that at least one duplicate number must exist in `nums`?
- Can you solve the problem in linear runtime complexity?

Q. **Valid Anagram**

Given two strings `s` and `t`, return `true` *if* `t` *is an anagram of* `s`*, and* `false` *otherwise*.

An **Anagram** is a word or phrase formed by rearranging the letters of a different word or phrase, typically using all the original letters exactly once.

Example 1:

```
Input: s = "anagram", t = "nagaram"

Output: true
```

Example 2:

```
Input: s = "rat", t = "car"

Output: false
```

Constraints:

- `1 <= s.length, t.length <= 5 * 10⁴`
- `s` and `t` consist of lowercase English letters.

Follow up: What if the inputs contain Unicode characters? How would you adapt your solution to such a case?

Q. **Minimum Cost to Move Chips to The Same Position**

Easy

1747226Add to ListShare

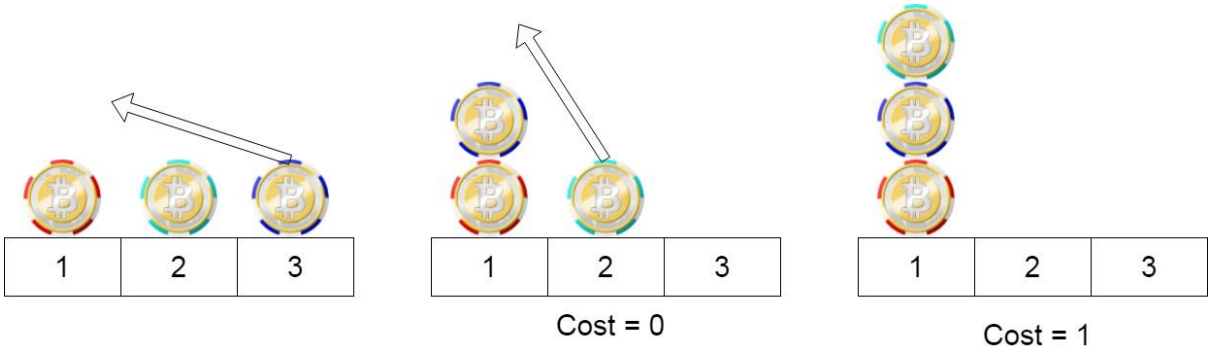
We have `n` chips, where the position of the `i`th chip is `position[i]`.

We need to move all the chips to **the same position**. In one step, we can change the position of the `i`th chip from `position[i]` to:

- `position[i] + 2` or `position[i] - 2` with `cost = 0`.
- `position[i] + 1` or `position[i] - 1` with `cost = 1`.

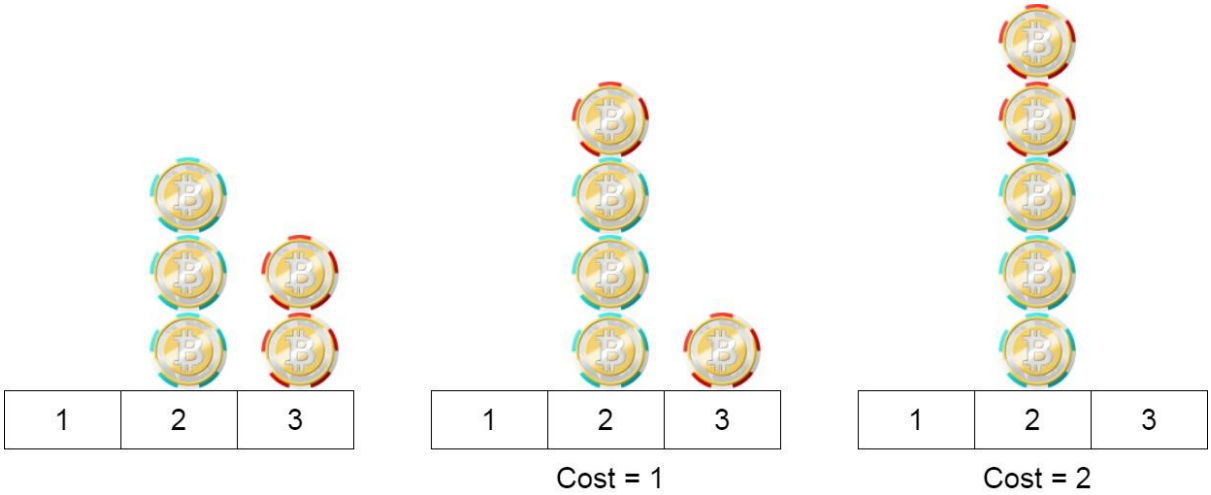
Return *the minimum cost* needed to move all the chips to the same position.

Example 1:




```
Input: position = [1,2,3]

Output: 1

Explanation: First step: Move the chip at position 3 to position 1 with cost = 0.

Second step: Move the chip at position 2 to position 1 with cost = 1.

Total cost is 1.
```

Example 2:




```
Input: position = [2,2,2,3,3]

Output: 2

Explanation: We can move the two chips at position  3 to position 2. Each move has
cost = 1. The total cost = 2.
```

Example 3:

```
Input: position = [1,1000000000]

Output: 1
```

Constraints:

- `1 <= position.length <= 100`
- `1 <= position[i] <= 10^9`

Q. **Find Peak Element**

A peak element is an element that is strictly greater than its neighbors.

Given a **0-indexed** integer array `nums`, find a peak element, and return its index. If the array contains multiple peaks, return the index to **any of the peaks**.

You may imagine that `nums[-1] = nums[n] = -∞`. In other words, an element is always considered to be strictly greater than a neighbor that is outside the array.

You must write an algorithm that runs in `O(log n)` time.

Example 1:

```
Input: nums = [1,2,3,1]

Output: 2
```

```
Explanation: 3 is a peak element and your function should return the index number 2.
```

Example 2:

```
Input: nums = [1,2,1,3,5,6,4]

Output: 5

Explanation: Your function can return either index number 1 where the peak element is 2, or index number 5 where the peak element is 6.
```

Constraints:

- `1 <= nums.length <= 1000`
- $-2^{31} <= nums[i] <= 2^{31} - 1$
- `nums[i] != nums[i + 1]` for all valid `i`.

Reference:- Leetcode 1, 1626, 146, 121, 103, 53, 50, 17, 287, 242, 1217, 162,

https://leetcode.com/discuss/interview-experience/130416/morgan-stanley-interview-experience

https://www.geeksforgeeks.org/morgan-stanley-interview-preparation/

CECGEU